

Project Documentation

Segue uma documentação detalhada para o seu projeto, com base no código fornecido e no padrão **Abstract Factory** aplicado:

Documentação do Projeto

1. Visão Geral

Este projeto implementa o padrão de projeto **Abstract Factory** para criar diferentes tipos de carros (SUVs e carros esportivos) e gerar relatórios sobre eles. O projeto segue os princípios **SOLID** para garantir um design modular, extensível e de fácil manutenção.

2. Estrutura do Projeto

A estrutura do projeto está organizada em namespaces e classes que seguem o padrão **MVP (Model-View-Presenter)** e o **Abstract Factory**. Abaixo está uma visão geral:

2.1. Namespaces Principais

- **AbstractFactory.Entities** : Contém as interfaces e classes que representam os produtos (SUVs e carros esportivos).
 - **AbstractFactory.Factories.Constructors** : Contém as fábricas abstratas e concretas para criar os produtos.
 - **AbstractFactory.MVP** : Implementa o padrão MVP, separando a lógica de apresentação, modelo e visualização.
 - **AbstractFactory.Services** : Contém os serviços responsáveis por criar relatórios e formatá-los.
-

3. Componentes do Projeto

3.1. Abstract Factory

O padrão **Abstract Factory** é usado para criar famílias de objetos relacionados (SUVs e carros esportivos) sem especificar suas classes concretas.

3.1.1. Interface da Fábrica

```
public interface IConstructorsFactory
{
    ISuv CreateSuv();
    ISportCar CreateSportCar();
}
```

- Define os métodos para criar SUVs e carros esportivos.

3.1.2. Fábricas Concretas

Exemplo de uma fábrica concreta:

```
public class FerrariFactory : IConstructorsFactory
{
    public ISuv CreateSuv() ⇒ new FerrariSuv();
    public ISportCar CreateSportCar() ⇒ new FerrariSportCar();
}
```

- Implementa a interface `IConstructorsFactory` para criar produtos específicos (Ferrari SUV e Ferrari Sport Car).

3.1.3. Produtos Abstratos

```
public interface ISuv
{
    string Model { get; }
    int HorsePower { get; }
}

public interface ISportCar
{
    string Model { get; }
    int HorsePower { get; }
}
```

- Define as propriedades comuns para SUVs e carros esportivos.

3.1.4. Produtos Concretos

Exemplo de um produto concreto:

```
public class FerrariSuv : ISuv
{
    public string Model ⇒ "Ferrari SUV";
    public int HorsePower ⇒ 450;
}
```

- Implementa a interface `ISuv` para representar um SUV específico.

3.2. MVP (Model-View-Presenter)

O padrão **MVP** é usado para separar a lógica de apresentação, o modelo e a visualização.

3.2.1. Model

A classe `MainModel` implementa a lógica de negócios:

```
public class MainModel(IConstructorsFactoryCreator constructorsFactoryC
reator, ICreateCarRelatory createCarRelatory) : IMainModel
{
    private readonly IConstructorsFactoryCreator _constructorsFactoryCreat
or = constructorsFactoryCreator;
    private readonly ICreateCarRelatory _createCarRelatory = createCarRelat
ory;

    public IConstructorsFactory GetConstructorFactoryByString(string const
ructorName)
        ⇒ _constructorsFactoryCreator.GetConstructorFactoryByString(const
ructorName);

    public Task CreateSuvRelatory(ISuv suv)
        ⇒ _createCarRelatory.CreateRelatorySuv(suv);

    public Task CreateSportCarRelatory(ISportCar sportCar)
```

```
⇒ _createCarRelatory.CreateRelatorySportCar(sportCar);  
}
```

- **Responsabilidade:** Delegar a criação de fábricas e relatórios.

3.2.2. View

A interface `IMainView` define os eventos e métodos da interface do usuário:

```
public interface IMainView  
{  
    string SelectedConstructor { get; }  
    event Action SaveSuv;  
    event Action SaveSportCar;  
    void ShowMessageBox(string message, string title, MessageBoxButtons  
        buttons, MessageBoxIcon icon);  
}
```

3.2.3. Presenter

A classe `MainPresenter` conecta a view e o modelo:

```
public class MainPresenter(IMainView view, IMainModel model)  
{  
    private readonly IMainView _view = view;  
    private readonly IMainModel _model = model;  
  
    public MainPresenter(IMainView view, IMainModel model)  
    {  
        _view.SaveSuv += CreateSuvRelatoryAsync;  
        _view.SaveSportCar += CreateSportCarRelatoryAsync;  
    }  
  
    private async void CreateSuvRelatoryAsync()  
    {  
        try  
        {  
            var factory = _model.GetConstructorFactoryByString(_view.Selected  
                Constructor);
```

```

        await _model.CreateSuvRelatory(factory.CreateSuv());
    }
    catch (Exception ex)
    {
        _view.ShowMessageBox(ex.Message, "Erro ao criar o relatório", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
}

```

- **Responsabilidade:** Gerenciar a interação entre a view e o modelo.

3.3. Serviços

Os serviços são responsáveis por criar e formatar relatórios.

3.3.1. Serviço de Relatórios

A classe `CreateCarRelatory` delega a criação de relatórios para o formatter:

```

public class CreateCarRelatory(IRelatoryCarFormatter relatoryCarFormatter) : ICreateCarRelatory
{
    private readonly IRelatoryCarFormatter _relatoryFormatter = relatoryCarFormatter;

    public Task CreateRelatorySuv(ISuv suv)
        => _relatoryFormatter.CreateRelatorySuv(suv);

    public Task CreateRelatorySportCar(ISportCar sportCar)
        => _relatoryFormatter.CreateRelatorySportCar(sportCar);
}

```

- **Responsabilidade:** Delegar a formatação de relatórios.

3.3.2. Formatter

A interface `IRelatoryCarFormatter` define os métodos para formatar relatórios:

```
public interface IRelatoryCarFormatter
{
    Task CreateRelatorySuv(ISuv suv);
    Task CreateRelatorySportCar(ISportCar sportCar);
}
```

Exemplo de implementação concreta:

```
public class RelatoryTextCarFormatter : IRelatoryCarFormatter
{
    public async Task CreateRelatorySuv(ISuv suv)
    {
        using var writer = new StreamWriter("SuvRelatory.txt");
        await writer.WriteLineAsync($"SUV Model: {suv.Model}, HorsePower: {suv.HorsePower}");
    }

    public async Task CreateRelatorySportCar(ISportCar sportCar)
    {
        using var writer = new StreamWriter("SportCarRelatory.txt");
        await writer.WriteLineAsync($"SportCar Model: {sportCar.Model}, HorsePower: {sportCar.HorsePower}");
    }
}
```

- **Responsabilidade:** Formatar relatórios em texto.

4. Fluxo de Execução

1. O usuário seleciona um construtor (Ferrari, Mercedes, etc.) na interface.
2. O `MainPresenter` captura o evento e solicita ao `MainModel` a fábrica correspondente.
3. O `MainModel` usa o `IConstructorsFactoryCreator` para obter a fábrica.
4. A fábrica cria o SUV ou carro esportivo.
5. O `MainModel` solicita ao `CreateCarRelatory` a criação do relatório.
6. O `CreateCarRelatory` delega a formatação ao `IRelatoryCarFormatter`.

7. O relatório é salvo no formato especificado (ex.: texto).

5. Princípios SOLID

O projeto segue os princípios SOLID:

- **SRP**: Cada classe tem uma única responsabilidade.
 - **OCP**: Novos tipos de carros ou relatórios podem ser adicionados sem modificar o código existente.
 - **LSP**: As abstrações permitem substituir implementações sem alterar o comportamento.
 - **ISP**: As interfaces são bem segregadas e específicas.
 - **DIP**: O projeto depende de abstrações, não de implementações concretas.
-

Com base no código fornecido e na estrutura geral do projeto, vamos analisar se ele segue os princípios **SOLID**:

1. Single Responsibility Principle (SRP)

Princípio: Cada classe deve ter uma única responsabilidade e uma única razão para mudar.

- **Análise**:
 - A classe `MainPresenter` é responsável por gerenciar a interação entre a view (`IMainView`) e o modelo (`IMainModel`). Ela também lida com eventos da interface do usuário, como salvar relatórios. Isso está de acordo com o SRP.
 - A classe `MainModel` é responsável por delegar a criação de fábricas e relatórios, o que também está alinhado com o SRP.
 - A classe `CreateCarRelatory` é responsável por delegar a formatação de relatórios para o formatter (`IRelatoryCarFormatter`), o que respeita o SRP.

Conclusão: O projeto segue o **SRP**, pois cada classe tem uma única responsabilidade bem definida.

2. Open/Closed Principle (OCP)

Princípio: As classes devem estar abertas para extensão, mas fechadas para modificação.

- **Análise:**

- O uso de interfaces como `IConstructorsFactory`, `ICreateCarRelatory` e `IRelatoryCarFormatter` permite que o comportamento do sistema seja estendido (por exemplo, adicionando novas fábricas ou formatadores) sem modificar as classes existentes.
- Se for necessário adicionar um novo tipo de veículo ou um novo formato de relatório, isso pode ser feito criando novas implementações das interfaces existentes, sem alterar o código das classes principais.

Conclusão: O projeto segue o **OCP**, pois está projetado para ser extensível sem exigir modificações nas classes existentes.

3. Liskov Substitution Principle (LSP)

Princípio: Subtipos devem ser substituíveis por seus tipos base sem alterar o comportamento do programa.

- **Análise:**

- As classes concretas que implementam interfaces como `IConstructorsFactory`, `ICreateCarRelatory` e `IRelatoryCarFormatter` podem ser substituídas por suas interfaces sem alterar o comportamento do sistema.
- Por exemplo, uma fábrica concreta como `FerrariFactory` pode ser substituída por outra implementação de `IConstructorsFactory` sem impactar o funcionamento do `MainPresenter`.

Conclusão: O projeto segue o **LSP**, pois as abstrações permitem a substituição de implementações concretas sem alterar o comportamento.

4. Interface Segregation Principle (ISP)

Princípio: Uma classe não deve ser forçada a depender de métodos que ela não utiliza.

- **Análise:**

- As interfaces no projeto, como `IMainView`, `IMainModel`, `IConstructorsFactory`, e `IRelatoryCarFormatter`, são bem segregadas e específicas para suas

responsabilidades.

- Por exemplo, `IMainView` define apenas os métodos e eventos necessários para a interação com a interface do usuário, e `IRelatoryCarFormatter` define apenas os métodos necessários para formatar relatórios.

Conclusão: O projeto segue o **ISP**, pois as interfaces são bem projetadas e específicas para suas responsabilidades.

5. Dependency Inversion Principle (DIP)

Princípio: Dependenda de abstrações, não de implementações.

- **Análise:**

- O projeto depende de abstrações em vez de implementações concretas. Por exemplo:
 - `MainPresenter` depende de `IMainView` e `IMainModel`.
 - `CreateCarRelatory` depende de `IRelatoryCarFormatter`.
 - `MainModel` depende de `IConstructorsFactoryCreator` e `ICreateCarRelatory`.
- Isso facilita a substituição de implementações concretas por outras (por exemplo, em testes unitários ou mudanças de requisitos).

Conclusão: O projeto segue o **DIP**, pois todas as dependências são baseadas em abstrações.

Resumo da Análise

Princípio	Seguido?	Observações
SRP	✓ Sim	Cada classe tem uma única responsabilidade bem definida.
OCP	✓ Sim	O projeto é extensível sem exigir modificações nas classes existentes.
LSP	✓ Sim	As abstrações permitem a substituição de implementações concretas sem problemas.
ISP	✓ Sim	As interfaces são bem segregadas e específicas para suas responsabilidades.
DIP	✓ Sim	O projeto depende de abstrações, não de implementações concretas.

Conclusão Geral

O projeto segue bem os princípios **SOLID**. Ele é modular, extensível e fácil de manter. As responsabilidades estão bem distribuídas entre as classes, e o uso de abstrações garante flexibilidade e desacoplamento.